

Predictive Modeling Plan

Credit Delinquency Risk Model — Geldium × Tata iQ

Prepared for:	Tata iQ Analytics Team & Geldium Decision-Makers
Follows from:	Geldium EDA Report (Task 1)
Dataset:	500 customers × 19 variables Target: Delinquent_Account (binary)
Date:	June 2026

1. Model Logic & Approach

1.1 Chosen Model: Gradient Boosted Decision Trees (XGBoost)

The recommended model for Geldium's delinquency prediction task is a **Gradient Boosted Decision Tree (GBDT)** implemented via XGBoost. GBDTs build an ensemble of shallow decision trees sequentially, each correcting the errors of its predecessor, producing a highly accurate classifier that handles mixed data types (numeric and categorical), missing values, and class imbalance gracefully. For interpretability — a regulatory requirement in financial services — the model's outputs can be explained at the individual prediction level using SHAP (SHapley Additive exPlanations) values, making it suitable for both operational deployment and regulatory audit.

1.2 Top 5 Input Features

Rank	Feature	Type	Contribution to Prediction
1	Missed_Payments	Numeric (0–6)	Strongest direct behavioural signal; each additional missed payment materially raises delinquency probability.
2	Credit_Utilization	Numeric (0–1)	Captures borrower financial stress; utilisation >70% is an industry-standard elevated-risk threshold.
3	Debt_to_Income_Ratio	Numeric (10%–55%)	Measures repayment capacity relative to obligations; high DTI constrains ability to service debt.
4	Payment_History_Score (engineered)	Numeric (0–6)	Aggregate of Missed + Late payments across Month_1–6; captures deteriorating payment trajectory.
5	Employment_Status	Categorical (4 classes)	Unemployment directly reduces income stability and elevates default likelihood.

Note: Income and Loan_Balance are included as secondary features after imputation. Credit_Score is retained but treated cautiously given the EDA finding that it does not cleanly separate delinquent from non-delinquent customers in this dataset.

1.3 Modeling Pipeline

Stage	Description
1. Data Ingestion	Load cleaned dataset; apply imputed Income/Loan_Balance and standardised Employment_Status labels
2. Feature Engineering	• Encode Employment_Status with one-hot encoding • Create Payment_History_Score = count of Missed/Late across Month_1–6 • Apply log-transform to Income to reduce right skew
3. Train/Test Split	80/20 stratified split to preserve 16% delinquency rate in both sets.
4. Class Imbalance Handling	Apply SMOTE (Synthetic Minority Over-sampling Technique) on training set only, or use XGBoost's built-in scale_pos_weight parameter (~5.25 = 420/80).
5. Model Training	Train XGBoost classifier with cross-validation (5-fold stratified k-fold).
6. Threshold Tuning	Default 0.5 probability threshold adjusted to optimise F1/Recall trade-off for the delinquent class — false negatives are costlier than false positives.
7. Output	Per-customer delinquency probability score (0–1) + binary risk flag.

2. Model Justification

2.1 Why XGBoost over Alternatives?

Model	Strengths	Weaknesses	Fit for Geldium?
Logistic Regression	• Highly interpretable • Fast to train • Regulatory-friendly	• Assumes linear relationships • May miss complex interactions • Weaker on imbalanced data	Good baseline; limited predictive power
Decision Tree (single)	• Fully explainable • Handles non-linearity • No scaling needed	• Prone to overfitting • Unstable with small changes • Lower accuracy	Useful for rules; too fragile for production
XGBoost (GBDT) ✓	• High accuracy on tabular data • Handles missing values natively • SHAP explainability • Robust to imbalance	• More complex to tune • Slower inference vs. logistic • Requires careful CV	RECOMMENDED: best balance of performance + explainability
Neural Network	• Can model highly complex patterns • Scales to large datasets	• Black-box — poor regulatory fit • Needs large data volumes • Slow to retrain	Not suitable; overkill for 500 records

2.2 Business Justification

XGBoost is the recommended model for Geldium because it delivers the strongest predictive performance on structured tabular data — the dominant format in credit risk — while remaining explainable through SHAP values, a regulatory necessity under frameworks such as the Equal Credit Opportunity Act (ECOA) and

forthcoming AI transparency requirements. Unlike neural networks, XGBoost can be retrained quickly on updated monthly data and produces a calibrated probability score per customer, enabling Geldium's intervention teams to tier customers by risk level (high / medium / low) and target outreach efficiently. The model's native tolerance for missing values also reduces pre-processing overhead, and its `scale_pos_weight` parameter directly addresses the 84/16 class imbalance without the complexity of external resampling — ensuring that high-risk customers are not systematically misclassified as safe.

3. Model Evaluation Strategy

3.1 Core Performance Metrics

Metric	Target Value	Why It Matters for Geldium
AUC-ROC	≥ 0.80	Measures overall discrimination ability across all thresholds; the primary headline metric for credit risk models.
Recall (Sensitivity) for Delinquent Class	≥ 0.75	Prioritised over precision: missing a delinquent customer (false negative) is more costly than a false alarm.
F1 Score (Delinquent Class)	≥ 0.70	Balances precision and recall; critical given class imbalance where accuracy alone is misleading.
Precision	≥ 0.65	Limits unnecessary interventions on low-risk customers; protects customer experience and operational cost.
Brier Score	< 0.10	Evaluates calibration of probability scores — important if Geldium uses scores to set intervention thresholds.
Accuracy	Reported but not primary	With 84% non-delinquent baseline, a naïve model scores 84%; accuracy alone is not a reliable metric here.

3.2 Fairness & Bias Checks

Beyond predictive accuracy, the model must be assessed for discriminatory outcomes across protected customer segments. The following fairness checks should be conducted before deployment:

- **Demographic Parity:** Compare delinquency prediction rates across Location groups and Age brackets. Flag if any group is predicted delinquent at a rate more than 20% higher than the overall population rate without a corresponding difference in actual default rates.
- **Equal Opportunity:** Compare Recall (true positive rate) for the delinquent class across Employment_Status groups. The model should not be systematically worse at identifying genuine defaulters in any employment category.
- **Feature Audit:** Remove or anonymise Location if it acts as a proxy for protected characteristics (e.g., postcodes correlating with ethnicity or socioeconomic status).
- **SHAP Value Audit:** Inspect global SHAP plots to verify that the model's top drivers are financially legitimate variables (Missed_Payments, Credit_Utilization, DTI) and not demographic proxies.

- **Disparate Impact Ratio (DIR):** Calculate $DIR = (\text{adverse rate for protected group}) / (\text{adverse rate for reference group})$. A DIR below 0.80 (the 4/5ths rule) triggers mandatory model review under US fair lending guidance.

3.3 When to Retrain or Revise the Model

Trigger	Recommended Action
AUC-ROC drops below 0.75 on new monthly data	Retrain model on rolling 12-month window
Recall for delinquent class falls below 0.65	Lower classification threshold or add new behavioural features
DIR drops below 0.80 for any protected group	Conduct full bias audit; consider fairness-aware resampling
Data distribution shifts significantly (e.g., new economic shock)	Trigger re-validation; consider model refresh
Regulatory guidelines change (AI/ML in credit)	Review explainability documentation and SHAP outputs

4. Model Pseudocode

The following pseudocode outlines the core modeling logic. This is conceptual and illustrative — not executable code.

```
# 1. LOAD & PREPARE DATA
df = load('delinquency_dataset.xlsx')
df['Income'] = impute_by_group_median(df, group='Employment_Status')
df['Loan_Balance'] = impute_by_regression(df, predictors=['Income', 'DTI'])
df['Employment_Status'] = standardise_labels(df['Employment_Status'])

# 2. FEATURE ENGINEERING
df['Payment_Score'] = count_missed_or_late(df, cols=['Month_1', ..., 'Month_6'])
df['Income_log'] = log(df['Income'])
df = one_hot_encode(df, cols=['Employment_Status', 'Credit_Card_Type', 'Location'])

# 3. DEFINE FEATURES & TARGET
X = df[['Missed_Payments', 'Credit_Utilization', 'DTI', 'Payment_Score',
        'Employment_Status_', 'Income_log', 'Loan_Balance', 'Credit_Score', ...]]
y = df['Delinquent_Account']

# 4. TRAIN/TEST SPLIT (stratified)
X_train, X_test, y_train, y_test = stratified_split(X, y, test_size=0.20)

# 5. HANDLE CLASS IMBALANCE
X_train_res, y_train_res = SMOTE().fit_resample(X_train, y_train)

# 6. TRAIN XGB00ST
model = XGBoostClassifier(scale_pos_weight=5.25, n_estimators=300,
                           max_depth=5, learning_rate=0.05)
model.fit(X_train_res, y_train_res, eval_set=[(X_test, y_test)])
```

```
# 7. TUNE THRESHOLD
proba = model.predict_proba(X_test)[:,-1]
threshold = find_optimal_threshold(proba, y_test, metric='F1')
y_pred = (proba >= threshold).astype(int)

# 8. EVALUATE
print(AUC_ROC(y_test, proba))
print(classification_report(y_test, y_pred))
plot_shap_summary(model, X_test)
```

5. Summary

This plan outlines a **Gradient Boosted Decision Tree (XGBoost)** model as the recommended approach for predicting customer delinquency at Geldium. The model is trained on five primary features — Missed_Payments, Credit_Utilization, Debt_to_Income_Ratio, a 6-month Payment History Score, and Employment_Status — which were identified as the strongest risk indicators during EDA. XGBoost was selected over logistic regression and neural networks because it balances high predictive accuracy with SHAP-based interpretability, satisfying both operational and regulatory requirements. Model performance will be evaluated primarily on AUC-ROC (target ≥ 0.80) and Recall for the delinquent class (target ≥ 0.75), with mandatory fairness checks using the Disparate Impact Ratio to ensure equitable treatment across customer segments. The model should be retrained on a rolling basis as new monthly payment data becomes available, with automatic triggers for review if performance metrics fall below defined thresholds.